Out-of-Order Pipelines

Performance Equation

$$\frac{\#Programs}{\#Seconds} = \frac{\#Programs}{\#insts} * \frac{\#insts}{\#cycles} * \frac{\#cycles}{\#seconds}$$

$$= \frac{IPC * freq}{\#insts/program}$$

$$= \frac{IPC * freq}{\#insts}$$ (assume just 1 program)

- IPC is the number of instructions per cycle
- Let us loosely refer to the reciprocal of the time per program as the **performance**

So, what does performance depend on …

#instructions in the program

- Depends on the compiler

Frequency

- Depends on the transistor technology and the architecture
  - If we have more pipeline stages, then the time to traverse each stage reduces roughly proportionally
  - Given that each stage needs to be processed in one clock cycle, smaller the stage, higher the frequency
  - To increase the frequency, we simply need to increase the number of pipeline stages

IPC

- Depends on the architecture and the compiler

How to improve performance?

There are **3** factors:

- IPC, #instructions, and frequency
- #instructions is dependent on the compiler → not on the architecture

Let us look at IPC and frequency

IPC

- What is the IPC of an in-order pipeline?

1 if there are no stalls, otherwise < 1

Methods to increase IPC

- Forwarding
- Having more not-taken branches in the code
- Faster instruction and data memories

What about frequency?

What is frequency dependent on …

Frequency = 1 / clock period

Clock Period:

- 1 pipeline stage is expected to take 1 clock cycle
- Clock period = maximum latency of the pipeline stages

How to reduce the clock period?

- Make each stage of the pipeline smaller by increasing the number of pipeline stages
- Use faster transistors

Limits to Increasing Frequency

Assume that we have the fastest possible transistors

Can we increase the frequency to 100 GHz?



Reasons

Limits to increasing frequency - II

What does it mean to have a very high frequency?

Before answering, keep these facts in mind:

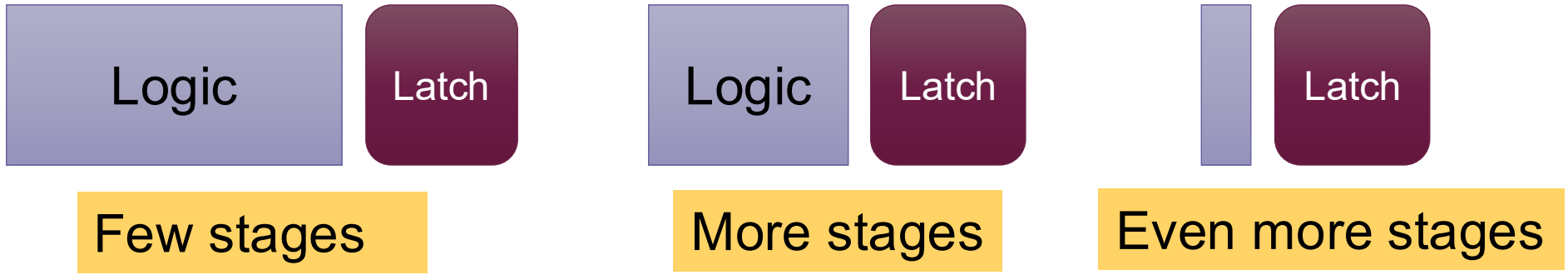
1. Thumb Rule: $P \propto f^3$

2. Thermo-dynamics: $\Delta T \propto P$

P → power
f → frequency
T → Temperature

3. We need to increase the number of pipeline stages → more hazards, more forwarding paths

How many pipeline stages can we have?



- We are limited by the latch delay
- Even with an infinite number of stages, the minimum clock period will be equal to the latch delay

Pipeline Stages vs IPC

$$CPI = \frac{1}{IPC}$$

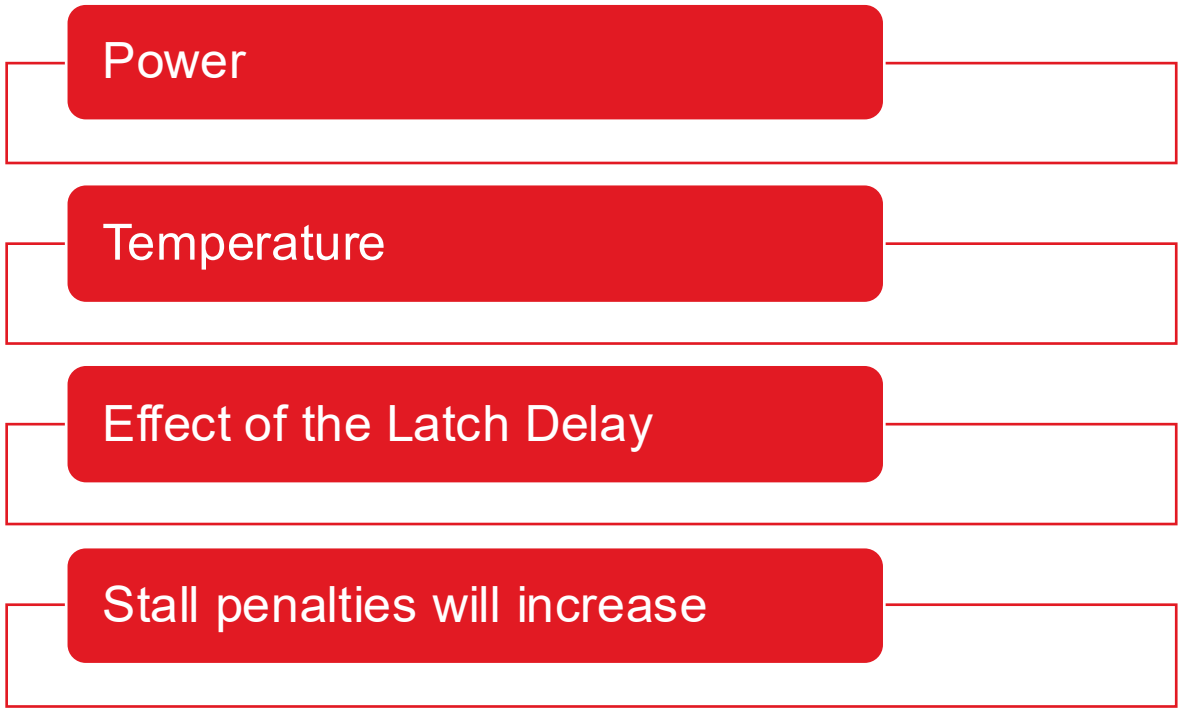
$$CPI = CPI_{ideal} + stall_rate * stall_penalty$$

- The stall rate will remain more or less constant per instruction with the number of pipeline stages
- The **stall penalty** (in terms of cycles) will however **increase**
- This will lead to a net increase in CPI and **loss** in IPC



As we increase the number of stages, the IPC goes down.

Summary: Why we cannot increase frequency by increasing the number of pipeline stages?



Since we cannot increase frequency …



Increase IPC

Increase IPC

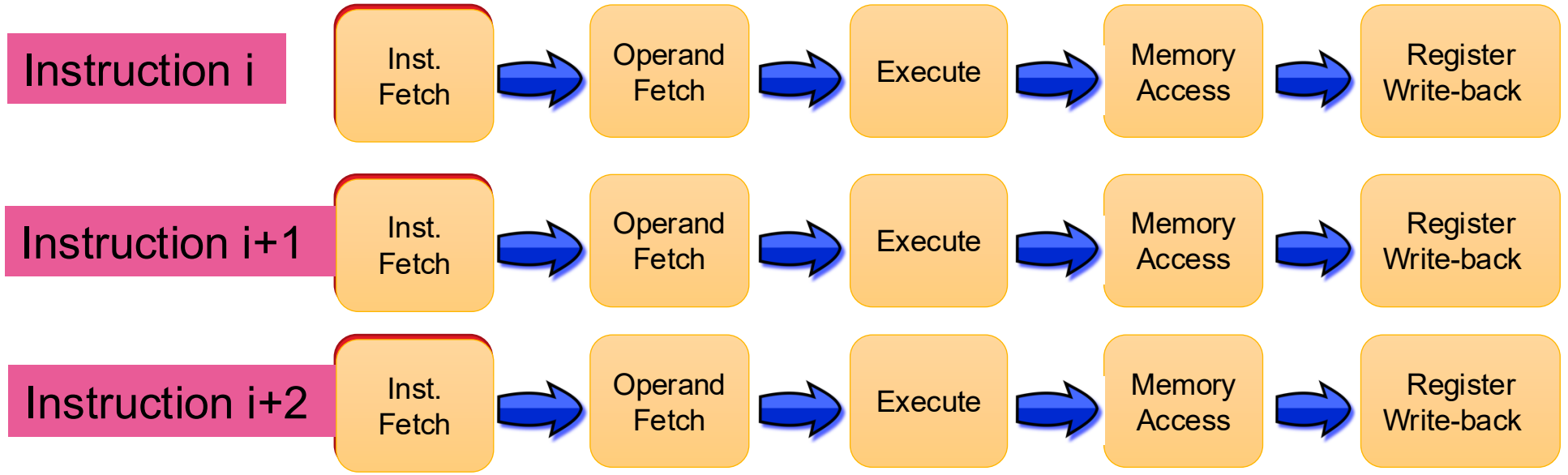
Issue more instructions per cycle

2, 4, or 8 instructions

Make it a **superscalar** processor → A processor that can execute multiple instructions per cycle

In-order Superscalar Processor

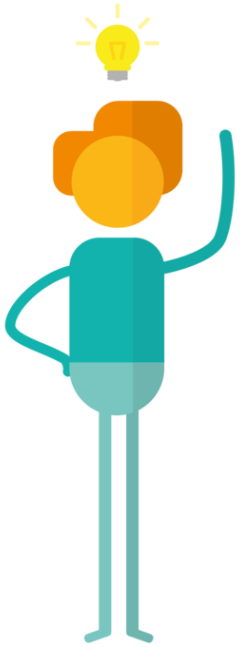
Have multiple in-order pipelines.



In-order Superscalar Processor - II

- There can be dependences between instructions
- Have $O(n^2)$ forwarding paths for an *n*-issue processor
- Complicated logic for detecting dependences, hazards, and forwarding
- Still might not be enough ...
- To get the peak IPC (= n) in an *n*-issue pipeline, we need to ensure that there are no stalls
- There will be no stalls if there are no taken branches, and no data dependences between instructions.
- Programs typically do not have such long sequences of instructions without dependences
- Load/store instn are difficult to handle

What to do ...



Don’t follow program order

```
mov r1, 1
add r3, r1, r2
add r4, r3, r2
mov r5, 1
add r6, r5, 1
add r8, r7, r6
```

Too many dependences

Execute out of order

Execute on a 2-issue OOO processor

```
mov r1, 1          mov r5, 1
add r3, r1, r2     add r6, r5, 1
add r4, r3, r2     add r8, r7, r6
```



Execute 2 instructions in parallel

Continuation ...

	issue slot 1	issue slot 2
cycle 1	mov r1, 1	mov r5, 1
cycle 2	add r3, r1, r2	add r6, r5, 1
cycle 3	add r4, r3, r2	add r8, r7, r6

In Out-of-order (OOO) processors, the execution is not as per program order. It is as per the data dependence order → the consumer is executed always after the producer.

Basic Principle of OOO Processors

Create a pool of instructions

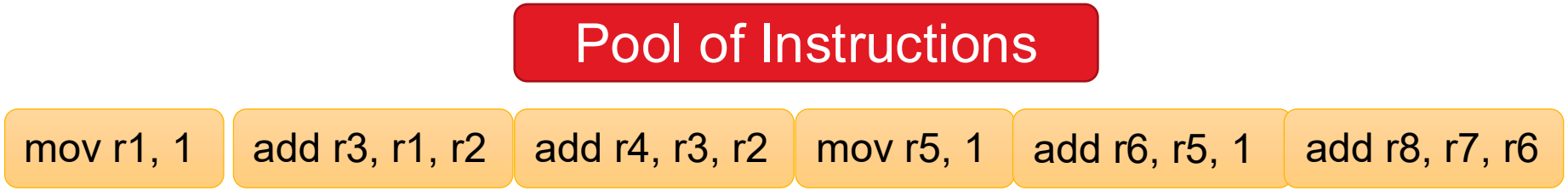
Find instructions that are mutually independent and have all their operands ready

Execute them out-of-order

ILP

Instruction level parallelism
The number of ready and independent instructions we can simultaneously execute.

Revisit the Example



Issue ready and mutually independent instructions

Pool of Instructions: Instruction Window

- Needs to be large enough such that the requisite number of mutually independent instructions can be found.
- Typical instruction window sizes: 64 to 128
- How do we create a large pool of instructions in a program with branches? We need to be sure that **all the instructions** are on the correct path



```
for (i = 1; i < m; i++) {
        for (j = 1; j < i; j ++ ) {
                if (j %2 == 0) continue;
                ....
        }
}
```

Example

Problems with creating an Instruction Pool

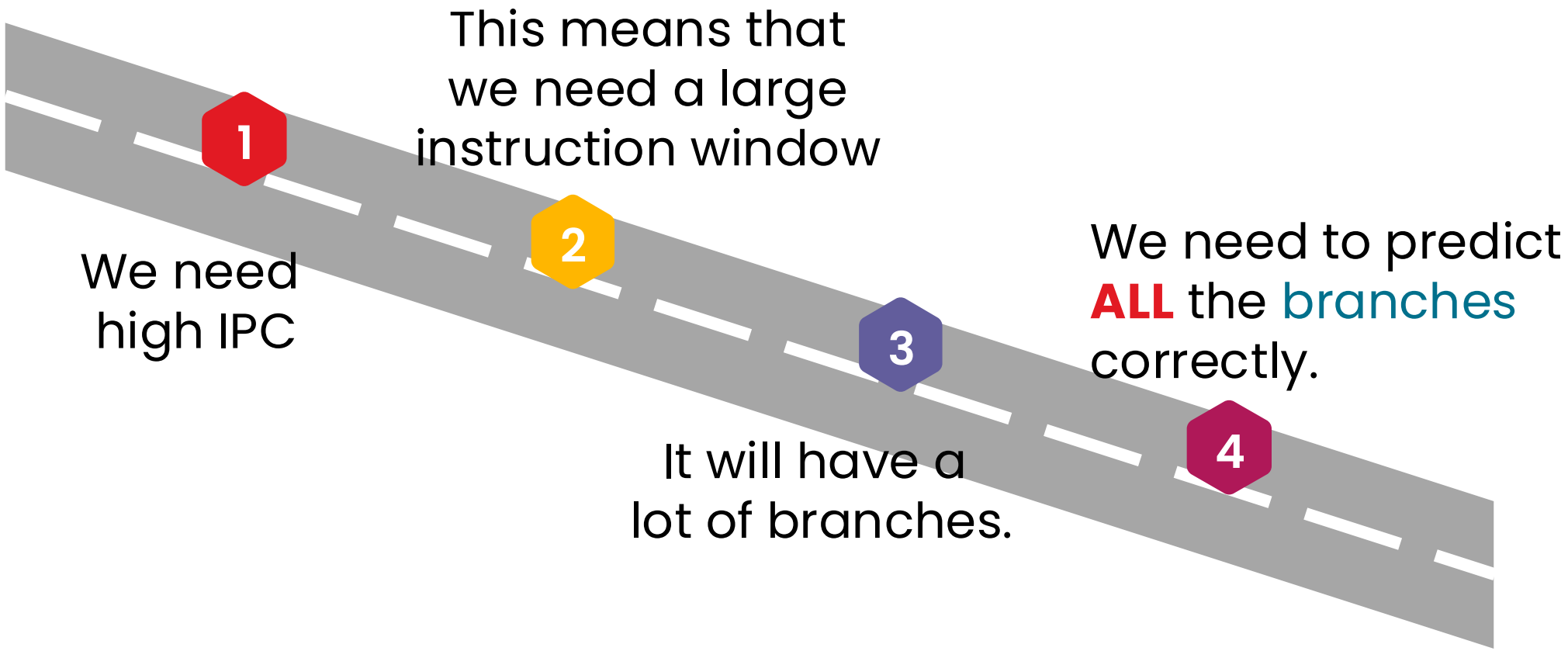


Typically 1 in 5 instructions is a branch



Predict the directions of the branches, and their targets

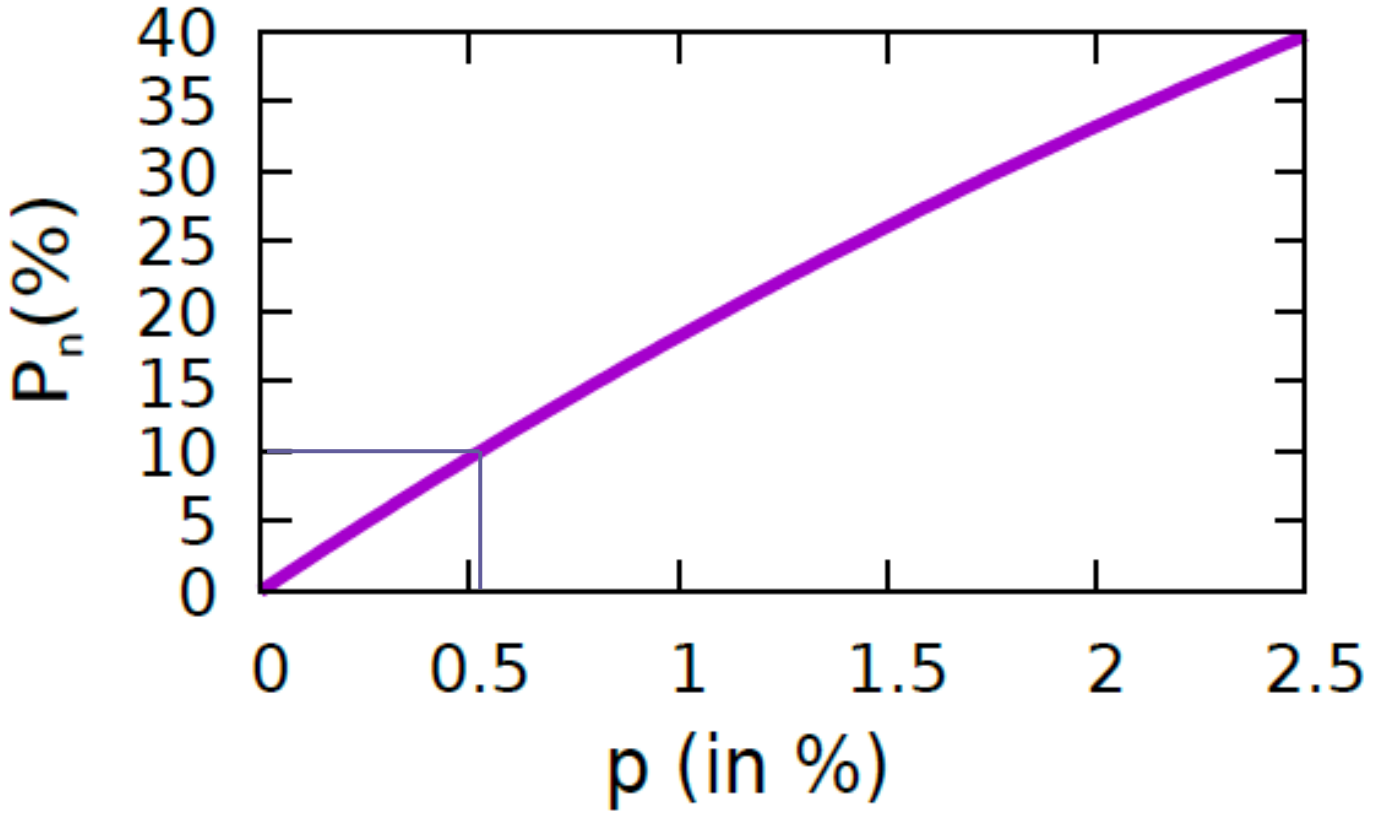
Motivation for Branch Prediction



The Maths of Branch Prediction

Number of instructions	n
Number of branches	**n/5**
Probability of predicting any given branch incorrectly	p
Probability of predicting **ALL the branches** correctly	$(1-p)^{\frac{n}{5}}$
Probability of making at least a single mistake (branch misprediction) in a pool of *n* instructions.	$P_n = 1-(1-p)^{\frac{n}{5}}$

For (n=100) : A plot of P_n vs p





If P_n = 10%, p has to be as low as 0.5% !!!



If we need a large instruction window, we need a very accurate branch predictor. The accuracy of the branch predictor limits the size of the instruction window.